# APEX PAY

*A Hybrid Financial Ecosystem for Android*

---

## Project Documentation

Software for Mobile Devices (CS-4039)


**Prepared By:** Abdul Moeed
Ahmad Siddique

**Submitted To:** Rana Waqas Ali

FAST NUCES

FAST National University of Computing and Emerging Sciences
Lahore Campus


**Spring 2026**

# Contents


**1 Introduction** **1**
1.1 Project Overview . . . 1
1.2 Problem Statement . . . 1
1.3 Objectives . . . 1
1.4 Scope . . . 2

**2 System Architecture** **3**
2.1 Architecture Pattern: MVVM . . . 3
2.2 Tech Stack . . . 3
2.3 High-Level System Flow . . . 4

**3 Project Structure** **6**
3.1 Root Package Layout . . . 6
3.2 Feature-Wise File Breakdown . . . 6
3.2.1 Authentication Module . . . 6
3.2.2 Digital Wallet Module . . . 7
3.2.3 Brokerage Module . . . 7
3.2.4 AI Module . . . 7
3.2.5 Analytics Module . . . 7
3.2.6 Profile and Settings . . . 7

**4 Detailed Feature Documentation** **8**
4.1 Multi-Tiered Authentication and Security . . . 8
4.2 Digital Wallet (P2P Transactions) . . . 8
4.3 Virtual Card and Subscription Tracker . . . 9
4.4 Simulated Retail Brokerage . . . 9
4.4.1 Mock Execution Engine . . . 9
4.5 AI Financial Intelligence . . . 9
4.5.1 Scan-to-Invest (Computer Vision) . . . 10
4.5.2 Risk Analyst Chatbot . . . 10
4.6 Visual Analytics Dashboard . . . 10
4.7 User Profile . . . 11

**5 Database Design** **12**
5.1 Overview . . . 12
5.2 Entity Relationship Diagram . . . 12
5.3 Table Schemas . . . 12
5.3.1 Users Table . . . 13
5.3.2 Transactions Table . . . 13
5.3.3 Portfolio (Holdings) Table . . . 14
5.3.4 Subscriptions Table . . . 14
5.4 Firebase Integration . . . 14

**6 API Integration** **16**
6.1 Financial Data APIs . . . . . 16
6.2 LLM API (AI Chatbot) . . . . . 16
6.3 Error Handling and Retry Logic . . . . . 16

**7 UI/UX Design** **18**
7.1 Design Philosophy . . . . . 18
7.2 Color Palette . . . . . 18
7.3 Navigation Structure . . . . . 18
7.4 Key Screens Summary . . . . . 19

**8 Testing Strategy** **20**
8.1 Unit Testing . . . . . 20
8.2 Integration Testing . . . . . 20
8.3 UI Testing . . . . . 20
8.4 Manual Testing . . . . . 20

**9 Setup Instructions** **22**
9.1 Prerequisites . . . . . 22
9.2 Clone and Open . . . . . 22
9.3 Firebase Setup . . . . . 22
9.4 API Keys Configuration . . . . . 23
9.5 Build and Run . . . . . 23

**10 Challenges and Limitations** **24**
10.1 Challenges Faced . . . . . 24
10.2 Known Limitations . . . . . 24

**11 Future Enhancements** **25**

**12 Conclusion** **26**

**References** **27**

# List of Figures

2.1 MVVM Architecture Overview . . . . . 3
2.2 High-Level System Flow Diagram . . . . . 5

4.1 Authentication Flow . . . . . 8

5.1 Entity Relationship Diagram . . . . . 12

7.1 Navigation Hierarchy . . . . . 19

# List of Tables


2.1 Technology Stack . . . . . 4

3.1 Package Descriptions . . . . . 6

5.1 Users Table Schema . . . . . 13
5.2 Transactions Table Schema . . . . . 13
5.3 Holdings Table Schema . . . . . 14
5.4 Subscriptions Table Schema . . . . . 14

6.1 API Endpoints Used . . . . . 16

7.1 Application Color Palette . . . . . 18
7.2 Screen Descriptions . . . . . 19

8.1 Testing Summary . . . . . 21

# Chapter 1

# Introduction

## 1.1 Project Overview

Apex Pay is a next-generation Android application that combines a peer-to-peer digital wallet, a simulated retail brokerage, and an artificial intelligence financial advisory layer into one unified platform. The app is built for students and young professionals who want to learn about personal finance, stock trading, and money management in a safe, simulated environment without risking real money.

The idea behind Apex Pay came from a simple observation: most finance apps either focus on payments or on investing, but never both. We wanted to build something that brings everything together under one roof, so a user does not have to jump between three or four different apps just to manage their money and learn about markets. Everything from sending cash to a friend, to buying fractional shares of Apple stock, to asking an AI chatbot whether Bitcoin is a good investment right now — it all happens inside one app.

## 1.2 Problem Statement

Currently, there is no single Android application in the market that provides a combined experience of digital payments, investment simulation, and AI-powered financial advice. Students who want to learn about stock trading have to use separate broker simulation apps, and those who want to manage their daily expenses use different wallet apps. This fragmented approach makes it harder for beginners to understand how their spending, saving, and investing habits are all connected.

Apex Pay solves this problem by putting all three functions into one clean interface, allowing users to see the full picture of their finances at a single glance.

## 1.3 Objectives

The primary objectives of the Apex Pay project are:

1. Build a fully functional P2P digital wallet that allows users to send and receive money locally.
2. Develop a simulated stock and crypto brokerage where users can practice trading with virtual money.
3. Integrate an AI chatbot that can analyze assets and provide basic risk assessments using financial models like Net Present Value (NPV) and Discounted Cash Flow (DCF).

4. Implement computer vision so users can scan product logos and instantly view related stock information.
5. Create a visually appealing dashboard with charts that help users track their portfolio and spending patterns.
6. Follow proper MVVM architecture with clean separation of concerns throughout the codebase.

## 1.4 Scope

The scope of Apex Pay covers the following areas:

- User authentication via Firebase (email/password) and biometric login (fingerprint-/face).
- A local ledger system using SQLite for offline-first data storage.
- Real-time market data fetching through public financial APIs.
- AI-powered chatbot using an LLM API for conversational financial analysis.
- Object recognition using Firebase ML Kit for the Scan-to-Invest feature.
- Visual analytics through donut charts and bar charts for portfolio and cash flow tracking.

**Disclaimer**

The application does not handle real money or connect to actual banking systems. All transactions and trades are simulated within the app for educational purposes only.

# Chapter 2

# System Architecture

## 2.1 Architecture Pattern: MVVM

Apex Pay follows the Model-View-ViewModel (MVVM) architecture pattern, which is the recommended architecture for modern Android development. The key advantage of MVVM is that each layer has a single responsibility, making the code easier to test, maintain, and scale over time.

- **Model:** Handles all data operations including SQLite database queries, Firebase calls, API requests, and local data classes (POJOs). The Model layer has no knowledge of the UI at all.
- **View:** The Activities and Fragments that the user interacts with. The View observes data from the ViewModel using LiveData and updates the UI accordingly. It does not contain any business logic.
- **ViewModel:** Acts as a bridge between the Model and the View. It holds UI-related data, processes user actions, and exposes results through LiveData objects. The ViewModel survives configuration changes like screen rotations, so the user does not lose data when they flip their phone.

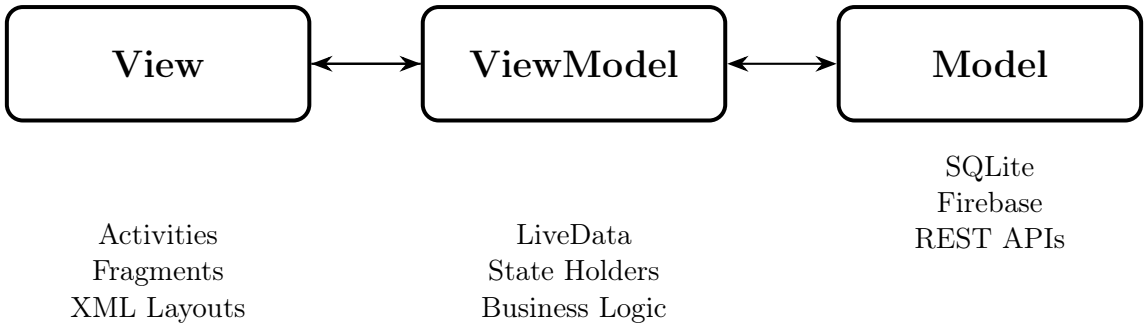


Figure 2.1: MVVM Architecture Overview

## 2.2 Tech Stack

Table 2.1 summarizes all the technologies used across different layers of the application.

Table 2.1: Technology Stack

| Component | Technology |
|---|---|
| Programming Language | Java |
| IDE | Android Studio |
| UI Framework | XML Layouts with custom Glassmorphic styling |
| Local Database | SQLite (via SQLiteOpenHelper) |
| Authentication | Firebase Authentication (Email/Password) |
| Machine Learning | Firebase ML Kit (On-device Object Recognition) |
| Market Data | Financial REST APIs (stocks, crypto, candlestick data) |
| AI Chatbot | LLM API (conversational risk analysis) |
| Charts | MPAndroidChart / Custom Canvas rendering |
| Architecture | MVVM with LiveData and ViewModel |
| Version Control | Git / GitHub |

## 2.3 High-Level System Flow

Figure 2.2 shows how data flows through the application from the moment the user opens it to the point where they interact with each major module.

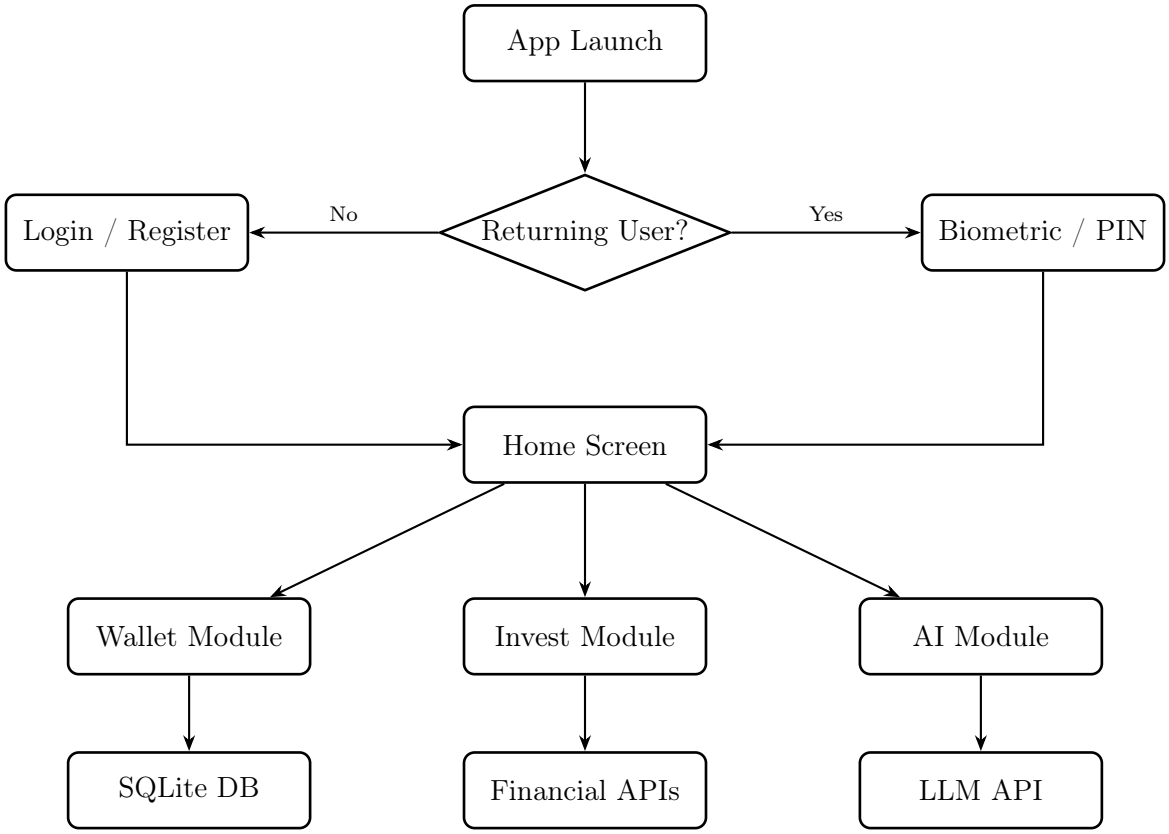


Figure 2.2: High-Level System Flow Diagram

# Chapter 3

# Project Structure

The project is organized into clearly defined packages following the MVVM pattern. Each major feature has its own sub-package containing its Model, View, and ViewModel components.

## 3.1 Root Package Layout

Table 3.1: Package Descriptions

| Package | Description |
|---|---|
| `activities/` | All Activity classes — entry points for each screen |
| `fragments/` | Reusable Fragment classes for modular UI sections |
| `viewmodels/` | ViewModel classes holding UI state and business logic |
| `models/` | Plain Java data classes (User, Transaction, Asset, Holding, etc.) |
| `adapters/` | RecyclerView and ListView adapters for displaying list data |
| `database/` | SQLiteOpenHelper subclass and DAO-style helper methods |
| `firebase/` | Firebase Authentication and Firebase ML integration classes |
| `api/` | Retrofit/HttpURLConnection service interfaces for market data and LLM |
| `utils/` | Utility and helper classes (date formatting, currency conversion, validators) |
| `widgets/` | Custom Views (frosted glass card, chart components, PIN pad) |

## 3.2 Feature-Wise File Breakdown

### 3.2.1 Authentication Module

- `LoginActivity.java`
- `RegisterActivity.java`
- `BiometricActivity.java`
- `PinActivity.java`
- `AuthViewModel.java`
- `AuthManager.java`

### 3.2.2 Digital Wallet Module

- `WalletFragment.java`
- `SendMoneyFragment.java`
- `TransactionHistoryFragment.java`
- `WalletViewModel.java`
- `Transaction.java`
- `TransactionAdapter.java`
- `FrequentContactAdapter.java`

### 3.2.3 Brokerage Module

- `MarketFragment.java`
- `AssetDetailFragment.java`
- `TradeFragment.java`
- `MarketViewModel.java`
- `TradeViewModel.java`
- `Asset.java`
- `Holding.java`
- `AssetListAdapter.java`
- `HoldingAdapter.java`
- `MarketApiService.java`

### 3.2.4 AI Module

- `AiChatFragment.java`
- `ScanInvestFragment.java`
- `AiChatViewModel.java`
- `ScanViewModel.java`
- `ChatMessage.java`
- `ChatAdapter.java`
- `LlmApiService.java`
- `MlKitManager.java`

### 3.2.5 Analytics Module

- `AnalyticsFragment.java`
- `AnalyticsViewModel.java`
- `DonutChartView.java`
- `CashFlowBarChart.java`

### 3.2.6 Profile and Settings

- `ProfileActivity.java`
- `SettingsFragment.java`
- `ProfileViewModel.java`

# Chapter 4

# Detailed Feature Documentation

## 4.1 Multi-Tiered Authentication and Security

The authentication system in Apex Pay has two layers. The first layer handles onboarding, where a new user registers with their email, password, username, and profile picture through Firebase Authentication. Upon successful registration, a unique account number in the format `APX-XXXX-XXXX-XXXX` is generated and stored locally in the SQLite database along with the user profile.

The second layer is for returning users. Instead of asking them to type their password every time, the app uses the AndroidX `BiometricPrompt` API to authenticate via fingerprint or face recognition. If biometrics are not available on the device, a four-digit PIN pad is displayed as a fallback. This approach keeps the login process quick while still maintaining strong security.

Internally, the `AuthViewModel` communicates with the `AuthManager` class which wraps Firebase calls. The ViewModel exposes a LiveData object that the `LoginActivity` observes. When authentication succeeds, the Activity navigates to the HomeActivity. If it fails, the error message is displayed through a Toast or AlertDialog.

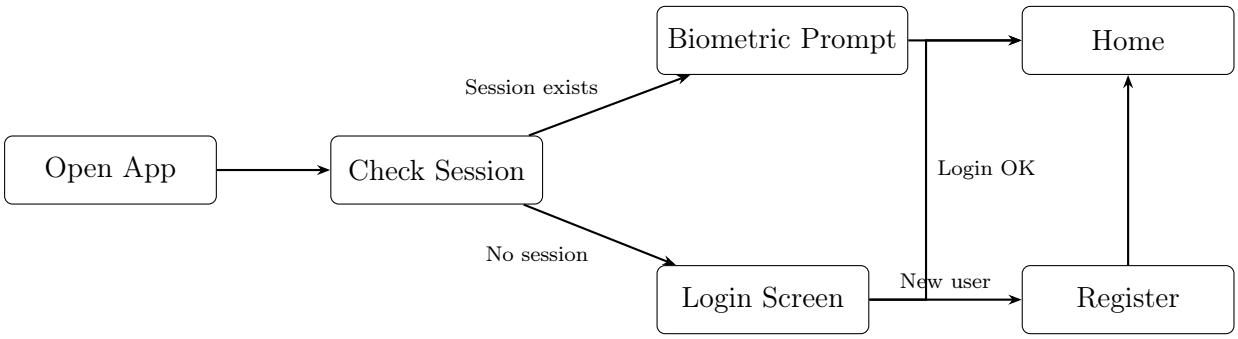


Figure 4.1: Authentication Flow

## 4.2 Digital Wallet (P2P Transactions)

The wallet module works like a simplified version of apps such as Venmo or CashApp. The user has a primary balance called **Liquid Cash**, stored in the local SQLite database. This balance can be topped up manually or through simulated bank deposits.

To send money, the user taps the Send button and either searches for a contact by username or account number, or selects someone from their frequent contacts row displayed at the top. The user enters an amount and an optional note, and the `WalletViewModel`

processes the transfer by deducting from the sender and crediting the receiver in a single SQLite transaction. This ensures data consistency even if the app crashes mid-operation.

The **Master Ledger** is a scrollable list showing all financial activity across the platform. It is backed by a `TransactionAdapter` that displays each entry with an icon, description, timestamp, and amount. Users can filter the ledger by transaction type (Sent, Received, Trades, Deposits) using chip buttons at the top of the screen.

## 4.3 Virtual Card and Subscription Tracker

The virtual card is purely a visual feature that shows a frosted-glass debit card tied to the user's account number. It displays the card number in a masked format, the cardholder name, and the expiry date. Users can toggle a Freeze/Unfreeze button which visually greys out the card and disables the Send Payment button in the wallet module.

Below the card, a subscription tracker lists the user's active monthly recurring expenses. Each item shows the service name (like Netflix or Spotify), the monthly cost, and the next billing date. This helps users keep track of how much money is leaving their account automatically each month, giving them a clearer picture of their fixed expenses.

## 4.4 Simulated Retail Brokerage

This is the investment module of the app. It fetches real-time market data from public financial APIs and displays categorized lists of assets including Stocks, Crypto, and Top Gainers. Each list item shows the asset name, current price, and daily percentage change with a green or red indicator.

Tapping on any asset opens an **Asset Detail** screen that presents an interactive candlestick chart for price history and a bar chart for daily trading volume. Core metrics such as Market Cap, P/E Ratio, 52-Week High, and 52-Week Low are also displayed. These values are fetched asynchronously using Retrofit and exposed through the `MarketViewModel`'s LiveData.

### 4.4.1 Mock Execution Engine

The Mock Execution Engine allows users to simulate buying and selling shares using their Liquid Cash. When the user places a buy order, the system calculates the total cost including:

- **Network Fee:** 0.1% of the order value
- **Spread:** 0.05% of the order value

The total is deducted from Liquid Cash, and the new holding is written to the Portfolio table in SQLite. The reverse happens on a sell order. A confirmation bottom sheet shows all the details before the trade is finalized.

## 4.5 AI Financial Intelligence

### 4.5.1 Scan-to-Invest (Computer Vision)

This feature uses Firebase ML Kit's on-device object recognition. When the user opens the camera from the AI tab and points it at a physical product, the ML model identifies the brand logo. The `ScanViewModel` then maps the recognized brand to its publicly traded ticker symbol using a local lookup table (for example, Nike maps to NKE, Apple maps to AAPL).

Once the ticker is identified, a bottom sheet slides up showing the current stock price, daily change, and a one-tap Buy button. This creates a fun and interactive way for users to discover investment opportunities in their everyday surroundings.

### 4.5.2 Risk Analyst Chatbot

The AI chatbot is a conversational interface where users can ask financial questions and get detailed responses powered by a large language model. The chat screen follows a familiar messaging layout with the user's messages on the right and the AI's responses on the left.

Users can attach specific asset data to their questions. For example, they can ask: "Should I invest more in BTC given current market volatility? What does the NPV analysis say?" The system sends the asset data along with the question to the LLM API, which responds with a structured risk assessment including:

- **Risk Level:** Low, Moderate, or High
- **12-Month NPV Estimate:** Projected value based on DCF analysis
- **Recommendation:** Hold, Buy, or Sell with a confidence percentage

The `AiChatViewModel` manages the conversation history and sends it as context with each API call so the AI can reference earlier messages in the same session.

## 4.6 Visual Analytics Dashboard

The analytics screen provides two key visualizations:

**Portfolio Distribution (Donut Chart):** A large multi-colored donut chart that shows how the user's investments are split across categories like Tech stocks, Crypto, and ETFs. The total profit or loss value is displayed in the center of the donut. This chart is drawn using a custom `DonutChartView` class that extends Android's View and uses Canvas drawing methods.

**Cash Flow Analysis (Bar Chart):** A grouped bar chart that tracks monthly cash inflow versus outflow. Green bars represent money coming in (deposits, received payments) and red bars represent money going out (purchases, sent payments, trades). This gives the user an instant visual comparison of their monthly budget health.

Both charts pull their data from the `AnalyticsViewModel`, which aggregates records from the SQLite transaction and portfolio tables.

## 4.7 User Profile

The profile screen shows the user's name, email, profile picture, account number, total net worth, transaction count, and account creation date. From this screen, users can access settings for:

- Editing their profile (name, photo, email)
- Managing linked bank accounts (simulated)
- Configuring notification preferences
- Updating biometric or PIN settings
- Managing privacy options

A Sign Out button at the bottom ends the session and returns to the login screen.

# Chapter 5

# Database Design

## 5.1 Overview

The local SQLite database is the backbone of Apex Pay's offline-first approach. All critical user data, including balances, transaction history, and portfolio holdings, are stored locally so the app works even without an internet connection. The database is managed through a custom `DatabaseHelper` class that extends `SQLiteOpenHelper`.

## 5.2 Entity Relationship Diagram

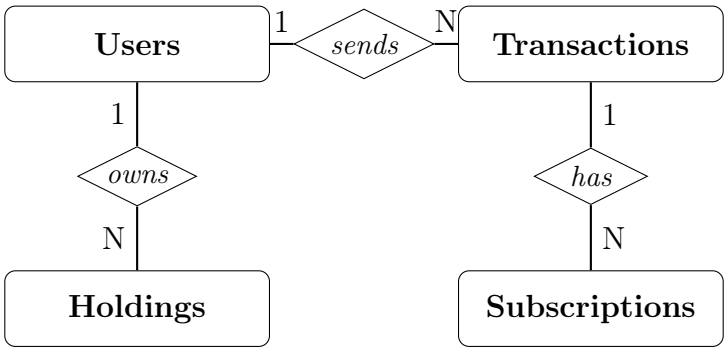


Figure 5.1: Entity Relationship Diagram

## 5.3 Table Schemas

### 5.3.1 Users Table

Table 5.1: Users Table Schema

| Column | Type | Constraint | Description |
|---|---|---|---|
| user_id | INTEGER | PK, AUTOINCREMENT | Unique user identifier |
| username | TEXT | UNIQUE, NOT NULL | Chosen display username |
| email | TEXT | UNIQUE, NOT NULL | Email linked to Firebase Auth |
| account_number | TEXT | UNIQUE, NOT NULL | System-generated (APX-XXXX-XXXX-XXXX) |
| balance | REAL | DEFAULT 0.0 | Liquid cash balance |
| profile_pic_path | TEXT | NULLABLE | Local path to profile image |
| pin_hash | TEXT | NOT NULL | Hashed 4-digit PIN |
| created_at | TEXT | DEFAULT CURRENT_TIMESTAMP | Account creation time |

### 5.3.2 Transactions Table

Table 5.2: Transactions Table Schema

| Column | Type | Constraint | Description |
|---|---|---|---|
| txn_id | INTEGER | PK, AUTOINCREMENT | Transaction ID |
| sender_id | INTEGER | FK (Users) | Who sent the money |
| receiver_id | INTEGER | FK (Users) | Who received the money |
| amount | REAL | NOT NULL | Transfer amount |
| txn_type | TEXT | NOT NULL | P2P, TRADE, or DEPOSIT |
| note | TEXT | NULLABLE | Optional user message |
| status | TEXT | DEFAULT ‘COMPLETED’ | Transaction status |
| timestamp | TEXT | DEFAULT CURRENT_TIMESTAMP | When it happened |

### 5.3.3 Portfolio (Holdings) Table

Table 5.3: Holdings Table Schema

| Column | Type | Constraint | Description |
|---|---|---|---|
| holding_id | INTEGER | PK, AUTOINCREMENT | Holding record ID |
| user_id | INTEGER | FK (Users) | Owner of the holding |
| ticker | TEXT | NOT NULL | Stock/crypto symbol (AAPL, BTC) |
| asset_name | TEXT | NOT NULL | Full asset name |
| quantity | REAL | NOT NULL | Number of shares or coins held |
| avg_buy_price | REAL | NOT NULL | Average purchase price |
| asset_type | TEXT | NOT NULL | STOCK, CRYPTO, or ETF |
| last_updated | TEXT | DEFAULT CURRENT_TIMESTAMP | Last modification time |

### 5.3.4 Subscriptions Table

Table 5.4: Subscriptions Table Schema

| Column | Type | Constraint | Description |
|---|---|---|---|
| sub_id | INTEGER | PK, AUTOINCREMENT | Subscription record ID |
| user_id | INTEGER | FK (Users) | Owner |
| service_name | TEXT | NOT NULL | e.g., Netflix, Spotify |
| monthly_cost | REAL | NOT NULL | Monthly charge amount |
| next_billing | TEXT | NOT NULL | Next billing date |
| is_active | INTEGER | DEFAULT 1 | 1 = active, 0 = cancelled |

## 5.4 Firebase Integration

Firebase is used for two specific purposes in Apex Pay and does not replace SQLite as the primary data store:

**Firebase Authentication:** Handles the initial account creation and email/password verification. When a user registers, Firebase creates the auth record and returns a UID. This UID is stored locally in the Users table to link the Firebase account with the local profile.

**Firebase ML Kit:** Provides on-device machine learning capabilities for the Scan-to-Invest feature. The ML model runs locally on the device, so it does not require an internet connection after the initial model download.

# Chapter 6

# API Integration

## 6.1 Financial Data APIs

The brokerage module fetches real-time data from public financial APIs. The following endpoints are consumed:

Table 6.1: API Endpoints Used

| Endpoint | Purpose |
|---|---|
| Stock Prices | Current price, daily high/low, percentage change for individual tickers |
| Crypto Prices | Real-time cryptocurrency prices in USD with 24-hour volume |
| Candlestick Data | OHLC data for a given ticker over a specified time range |
| Market Overview | Top gainers, top losers, and most active stocks |

All API calls are made asynchronously using either Retrofit with Gson converter or a custom `HttpURLConnection` wrapper. The `MarketViewModel` stores the results in LiveData objects, and the UI observes these to update the RecyclerView and chart components without blocking the main thread.

## 6.2 LLM API (AI Chatbot)

The AI chatbot communicates with a large language model through a REST API. Each message from the user is sent as part of a conversation array that includes the full chat history, so the model has context for follow-up questions. When asset data is attached, it is serialized into JSON and appended to the user's message as structured context.

The API response is parsed and displayed as a formatted message in the chat view. If the response contains structured data like risk levels or NPV values, these are rendered as a styled card within the chat bubble rather than plain text.

## 6.3 Error Handling and Retry Logic

Every API call in the app is wrapped in a try-catch block. If a network request fails due to a timeout or server error, the ViewModel sets an error state in its LiveData, and the Fragment shows a Snackbar with a Retry button.

For market data specifically, the app caches the last successful response in `SharedPreferences` so that the user still sees recent data even when offline. The cached data is marked with a timestamp, and a small banner at the top of the screen informs the user that they are viewing stale data.

**Caching Strategy**

Cached market data is valid for 30 minutes. After that, a refresh is triggered automatically when the user opens the Invest tab. If the refresh fails, the stale data remains visible with a "Last updated X minutes ago" label.

# Chapter 7

# UI/UX Design

## 7.1 Design Philosophy

The visual design of Apex Pay follows a dark-theme Glassmorphic style. The main background uses deep navy and dark gray tones, while interactive elements use frosted glass effects with subtle transparency. Accent colors are bright green (`#00C853`) for positive values and red (`#FF1744`) for negative values. Primary action buttons use vibrant green to stand out against the dark background.

The design was prototyped in Figma before being translated into XML layouts. Every screen was reviewed for consistency in spacing, typography, and color usage before implementation.

## 7.2 Color Palette

Table 7.1: Application Color Palette

| Color | Hex Code | Usage |
|---|---|---|
| Dark Background | `#0D1117` | Main screen backgrounds |
| Card Background | `#161B22` | Cards, panels, bottom sheets |
| Success Green | `#00C853` | Gains, positive values, CTA buttons |
| Danger Red | `#FF1744` | Losses, negative values, warnings |
| Accent Blue | `#2E75B6` | Links, highlights, secondary actions |
| White Text | `#FFFFFF` | Primary text on dark backgrounds |
| Gray Text | `#8B949E` | Secondary text, timestamps, labels |

## 7.3 Navigation Structure

The app uses a **Bottom Navigation Bar** with five tabs: Home, Invest, Wallet, AI, and Profile. Each tab loads its corresponding Fragment into the main content area. Some screens, like Asset Detail and Mock Trade, are opened as separate Activities since they represent deeper navigation paths that should not be part of the bottom tab flow.

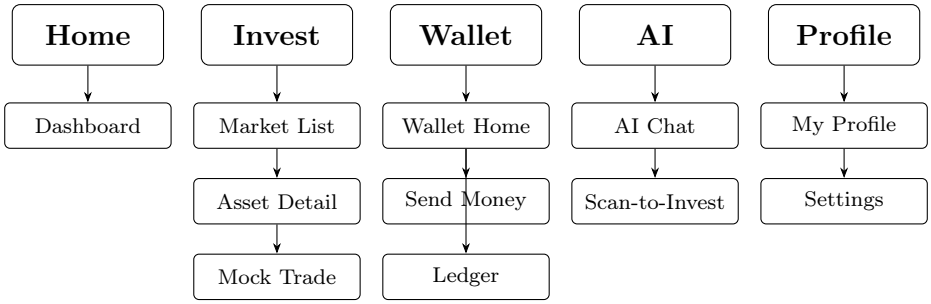


Figure 7.1: Navigation Hierarchy

## 7.4 Key Screens Summary

Table 7.2: Screen Descriptions

| Screen | Content |
|---|---|
| Home | Total net worth, allocation donut, cash flow chart, recent transactions |
| Invest | Categorized asset lists (Stocks, Crypto, Top Gainers) with live prices |
| Asset Detail | Candlestick chart, volume bars, key metrics, Buy/Sell buttons |
| Mock Trade | Order form with quantity selector, fee breakdown, confirmation |
| Wallet | Virtual card, liquid cash, quick send contacts, send payment form |
| Master Ledger | Full transaction history with type filters |
| AI Chat | Chatbot interface with asset attachment and risk analysis cards |
| Scan-to-Invest | Camera view with brand recognition and one-tap buy sheet |
| Profile | User info, account stats, settings menu, sign out |
| Register / Login | Account creation, email/password login, biometric prompt, PIN fallback |

# Chapter 8

# Testing Strategy

Testing is performed at multiple levels to make sure every part of the application works correctly before final submission.

## 8.1 Unit Testing

Unit tests are written for ViewModel classes and utility methods using JUnit 4. For example, the `WalletViewModel`'s transfer logic is tested to verify that the sender's balance decreases and the receiver's balance increases by the exact transfer amount. Edge cases like insufficient balance, zero amount, and self-transfer are also covered.

## 8.2 Integration Testing

Integration tests verify that the ViewModel correctly interacts with the SQLite database. These tests use an in-memory SQLite database (created with the context from `AndroidJUnit4`) to avoid polluting the real database. They check that transactions are written correctly, portfolio updates are atomic, and queries return the expected results.

## 8.3 UI Testing

UI tests are written using Espresso for critical user flows like registration, login, sending money, and placing a mock trade. Each test simulates user input (typing text, clicking buttons, scrolling lists) and asserts that the correct result is displayed on screen.

## 8.4 Manual Testing

Manual testing is done for features that are harder to automate, such as the biometric prompt, camera-based scanning, and chart rendering. A testing checklist is maintained with scenarios for different device screen sizes and Android versions (API 26 through API 34).

Table 8.1: Testing Summary

| **Level** | **Tool** | **Scope** |
|---|---|---|
| Unit | JUnit 4 | ViewModels, utility methods, edge cases |
| Integration | AndroidJUnit4 | ViewModel + SQLite interaction, data consistency |
| UI | Espresso | Registration, login, send money, mock trade flows |
| Manual | Checklist | Biometrics, camera, charts, multi-device testing |

# Chapter 9

# Setup Instructions

## 9.1 Prerequisites

- Android Studio Hedgehog (2023.1.1) or later
- Java Development Kit (JDK) 17
- Android SDK with minimum API level 26 (Android 8.0)
- A physical Android device or emulator running API 26+
- A Firebase project with Authentication and ML Kit enabled
- API keys for the financial data provider

## 9.2 Clone and Open

1. Clone the repository from GitHub:

```
git clone https://github.com/Moeeeeed/apexpay.git
```

2. Open Android Studio and select **Open an Existing Project**.
3. Navigate to the cloned folder and select it.
4. Wait for Gradle to sync all dependencies (may take a few minutes on first run).

## 9.3 Firebase Setup

1. Go to the Firebase Console (`console.firebase.google.com`) and create a new project named `ApexPay`.
2. Add an Android app with your package name (`com.apexpay`) and download the `google-services.json` file.
3. Place the `google-services.json` file in the `app/` directory of the project.
4. In the Firebase Console, enable **Email/Password** authentication under the Authentication section.
5. Enable **ML Kit** under the Machine Learning section for Scan-to-Invest functionality.

## 9.4 API Keys Configuration

Create or edit the file `local.properties` in the project root and add:

```
MARKET_API_KEY=.....
LLM_API_KEY=.....
```

These keys are read at build time through the `BuildConfig` class and are never hardcoded in source files.

## 9.5 Build and Run

1. Connect a physical device via USB or start an emulator from the AVD Manager.
2. Select the device from the target dropdown in Android Studio.
3. Click the green **Run** button or press `Shift+F10`.
4. The app should install and launch. Register a new account to start exploring.

**First Run Note**

On the first launch, Firebase ML Kit may need to download the object recognition model. Make sure your device has an active internet connection during the initial setup.

# Chapter 10

# Challenges and Limitations

## 10.1 Challenges Faced

**Glassmorphic UI in XML**
Achieving the frosted glass effect in native Android XML layouts was not straightforward. We had to use a combination of custom backgrounds with semi-transparent gradients, rounded corners, and subtle blur effects using RenderScript on devices that support it.

**Offline-First with Real-Time Data**
Balancing offline functionality (SQLite) with real-time market data required careful state management. We implemented a caching layer that stores the last known prices and flags stale data in the UI when the network is unavailable.

**Camera Permission Handling**
Android's runtime permission model (API 23+) required careful handling for the Scan-to-Invest feature. We had to handle all the edge cases: permission denied, permission permanently denied (showing a dialog to open Settings), and camera not available on the device.

**LLM Response Formatting**
The responses from the LLM API sometimes came back in different formats (plain text vs. structured JSON). We built a parser in the `AiChatViewModel` that tries to extract structured data first and falls back to plain text rendering if parsing fails.

## 10.2 Known Limitations

- All transactions are simulated — no real money is involved and no actual bank integration exists.
- The Scan-to-Invest feature depends on Firebase ML Kit's general object recognition, which may not accurately identify all brand logos. The local lookup table currently supports approximately 50 popular brands.
- Market data depends on the free tier of financial APIs, which may have rate limits and delayed quotes (15-minute delay for some providers).
- The app is designed for phones only and has not been optimized for tablet layouts.
- The LLM chatbot requires an active internet connection and cannot function offline.

# Chapter 11

# Future Enhancements

While the current version of Apex Pay covers all the planned features, there are several improvements we would like to add in future iterations:

1. **Social Feed:** A community feed where users can share their portfolio performance and trade insights with other Apex Pay users.
2. **Push Notifications:** Alerts for price movements, completed trades, and incoming P2P payments using Firebase Cloud Messaging.
3. **Multi-Currency Support:** Allowing users to hold and convert between different fiat currencies within the wallet.
4. **Dark/Light Theme Toggle:** While the current app uses a dark theme by default, adding a light mode option would improve accessibility for users who prefer brighter interfaces.
5. **Advanced Charting:** Adding technical indicators like Moving Averages, RSI, and Bollinger Bands to the candlestick chart for more advanced users.
6. **Tablet Layout:** Creating responsive layouts that take advantage of larger screen real estate on tablets.
7. **Portfolio Export:** Allowing users to export their transaction history and portfolio summary as a PDF or CSV file for record keeping.

# Chapter 12

# Conclusion

Apex Pay demonstrates that it is possible to build a feature-rich, multi-purpose financial application using native Android development tools. By combining a digital wallet, an investment simulator, and an AI-powered advisory layer into a single app, we have created a platform that helps users understand the full spectrum of personal finance in an interactive and engaging way.

The project follows the MVVM architecture pattern strictly, with every feature organized into its own package containing dedicated Model, View, and ViewModel components. The codebase is structured for clarity and maintainability, with meaningful comments explaining the reasoning behind important design decisions.

Building Apex Pay has been a valuable learning experience. We got hands-on practice with real-time API integration, local database management, computer vision, conversational AI, and modern Android UI design patterns. These are skills that directly translate to professional mobile development work, and we are confident that this project showcases our ability to design, architect, and implement a complex Android application from scratch.

# References


1. Android Developers. *Guide to App Architecture*. https://developer.android.com/topic/architecture
2. Android Developers. *ViewModel Overview*. https://developer.android.com/topic/libraries/architecture/viewmodel
3. Android Developers. *LiveData Overview*. https://developer.android.com/topic/libraries/architecture/livedata
4. Firebase Documentation. *Firebase Authentication*. https://firebase.google.com/docs/auth
5. Firebase Documentation. *ML Kit for Android*. https://firebase.google.com/docs/ml-kit
6. Android Developers. *BiometricPrompt*. https://developer.android.com/reference/androidx/biometric/BiometricPrompt
7. Android Developers. *SQLite Database Guide*. https://developer.android.com/training/data-storage/sqlite
8. Square Inc. *Retrofit — A Type-Safe HTTP Client for Android*. https://square.github.io/retrofit/
9. PhilJay. *MPAndroidChart Library*. https://github.com/PhilJay/MPAndroidChart
10. Android Developers. *Espresso Testing Framework*. https://developer.android.com/training/testing/espresso